ECE 437 - Lab 9 Pre Lab

Tong Li

Question 1: The SPI frame

The frame is 16 bits, MSB first: one control bit, seven address bits, eight data bits.

a. Give, in hex, the 16-bit word that reads register 83, and the word that writes the value 251 to register 83.

a. Reg 83 = 0x53 goes in bits [14:8]; 251 = 0xFB.
Read reg 83: 0 | 1010011 | 00000000 = **0x5300**
Write 251 to reg 83: 1 | 1010011 | 11111011 = **0xD3FB**

b. The sensor samples SPI_IN on the rising edge of SPI_CLK and launches SPI_OUT on the falling edge. State which edge your master should drive SPI_IN on, and which edge it should sample SPI_OUT on. Justify both from the sensor's behaviour, not from convention.

b. Drive SPI_IN on the **falling** edge: the sensor samples on the rising edge, so changing the bit half a period earlier gives it full setup and hold time around that rising edge.
Sample SPI_OUT on the **rising** edge: the sensor launches the bit on the falling edge, so by the next rising edge it has settled for half a period and will not change until the next falling edge.

c. The register defaults include reg 1 = 232 and reg 2 = 1, the low and high bytes of Number_lines. Reconstruct it, and say what physical property of the sensor that number is. Then: if your shift register were off by one bit on reads, what would you compute instead, and would you notice?

c. Number_lines = 232 + 256 x 1 = **488**, the number of rows (vertical resolution) of the pixel array.
Off by one bit (shift one extra): 232 -> 208, 1 -> 2, so 208 + 512 = **720**. Off by one bit (one short): 232 -> 116, 1 -> 0, so **116**.
Reg 1 alone (208 or 116) looks like a plausible byte, so it would go unnoticed; the composite is not 488, so checking the composite against the known row count catches it.

Question 2: Derive the timing, in both regimes

CLK_IN is 25 MHz. From the datasheet: a row is 324 pixels, one invalid slot, then 324 pixels, with LVAL low for one clock between rows, so budget 650 CLK_IN periods per row. The frame overhead time with reg 58 at its default is 4875 clocks.

a. Compute, in milliseconds: one row, the FOT, the readout of a 488-row frame, and the exposure the default Exp_time gives.

a. T = 1/25 MHz = 40 ns. Default Exp_time = 488 (regs 42/43 = 232, 1); datasheet: t_exp = Exp_time x 325 x T.

Quantity	Calculation	Result
One row	650 x 40 ns	0.026 ms
FOT	4875 x 40 ns	0.195 ms
Readout (488 rows)	488 x 0.026 ms	12.688 ms
Exposure (default)	488 x 325 x 40 ns	6.344 ms

b. One frame, from idle. The sensor exposes, then does the FOT, then reads out, in that order. How long is the silence between your FRAME_REQ pulse and the first LVAL, and how long from the first LVAL to the last?

b. One frame from idle:
Silence (FRAME_REQ -> first LVAL) = exposure + FOT = 6.344 + 0.195 = **6.54 ms**

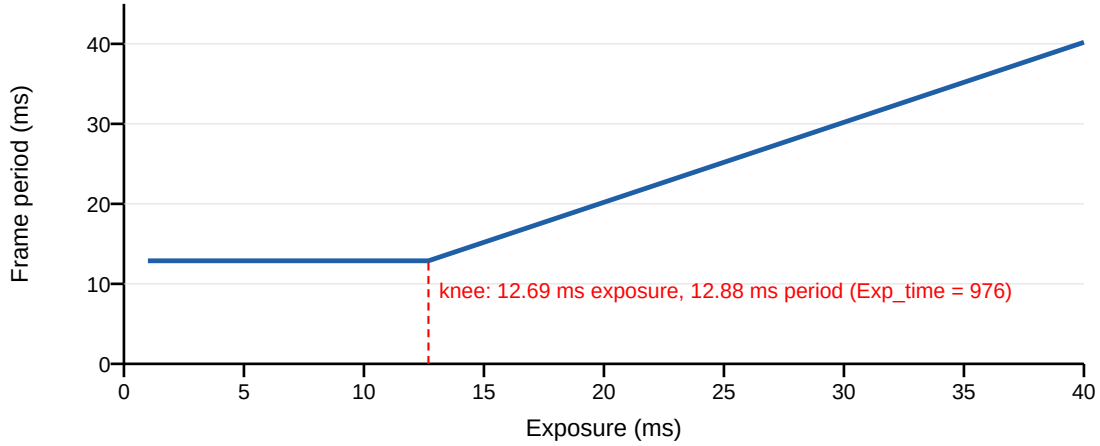
First LVAL -> last LVAL = readout = **12.69 ms** (12.66 ms to the last LVAL rising edge)
Total single frame = 6.344 + 0.195 + 12.688 = **19.23 ms** (about 52 fps)

c. Pipelined. With Number_frames greater than 1 the next exposure overlaps the current readout, so the frame period is FOT + max(readout, exposure). Compute it at the default exposure, give the ratio to your answer in (b), and explain in one or two sentences what pipelining actually saves.

c. Pipelined period = FOT + max(readout, exposure) = 0.195 + max(12.688, 6.344) = **12.88 ms** (77.6 fps).
Ratio = 12.88 / 19.23 = **0.67** (single-frame is 1.49x longer).
Pipelining does not make readout faster; it hides the exposure inside the previous frame's readout, so the exposure time is no longer paid separately.

d. Sketch pipelined frame period against exposure from 1 ms to 40 ms. At what exposure is the knee, and what Exp_time value is that?

d. Period = 0.195 ms + max(12.688 ms, t_exp). Flat at 12.88 ms until exposure equals readout, then rises with slope 1 (40 ms -> 40.2 ms).
Knee at t_exp = **12.69 ms** -> Exp_time = 12.688 ms / (325 x 40 ns) = **976**.



Question 3: The preconditions and the errata

a. The PLL bands are 251 for 10-32 MHz, 187 for 20-50 MHz and 155 for 32-50 MHz. The power-on default for reg 83 is 155 and your CLK_IN is 25 MHz. Which value should it be, and what does the datasheet say? With the wrong band the PLL still locks and the CLK_OUT meter reads about 36,700 cycles per millisecond instead of 25,000 - by what percentage would every timing measurement then be wrong?

a. CLK_IN = 25 MHz is in the 10-32 MHz band, so reg 83 must be **251**. The datasheet gives a power-on default of 155 (the high-frequency band) and says reg 83 must be reprogrammed to the correct range whenever CLK_IN changes, so the default is wrong for this board.
Error with the wrong band: 36,700 / 25,000 = 1.468 -> every timing measurement off by about **47%**.

b. List the five preconditions from the Lab 9 page that must all hold before FRAME_REQ does anything. For each, state the one counter reading that would reveal it was not met.

b.

Precondition	Counter reading that reveals it failed
CLK_IN running and PLL band correct (reg 83 = 251)	CLK_OUT meter = 0, or about 36,700 instead of 25,000 cycles/ms
reg 69 = 9 (CLK_OUT enabled)	CLK_OUT meter = 0 (and no LVAL edges)

Number_frames >= 1	CLK_OUT = 25,000 but LVAL row count stays 0 after FRAME_REQ
SYS_RES_N released >= 1 us after CLK_IN starts	SPI read-back of Number_lines (reg 1, 2) is not 488
>= 1 ms after SPI upload before FRAME_REQ	First frame row count is not 488 (missing or short frame)

c. Two of the five are timing requirements measured in microseconds. Explain in one or two sentences why they must be enforced by hardware in your sequencer rather than by a sleep in Python. Then: FRAME_REQ must be a pulse, not a level. Describe what a held-high FRAME_REQ produces, and which of the two regimes from question 2 it puts the sensor into.

c. The 1 us waits (clock -> reset release, reset release -> SPI) must be counted in CLK_IN cycles by the FPGA sequencer: Python sleeps and USB transfers have millisecond-scale, non-deterministic timing and cannot place an edge relative to CLK_IN with microsecond accuracy or guaranteed order.
A held-high FRAME_REQ re-triggers continuously, so the sensor produces frames forever (free-run). This puts it in the **pipelined** regime: one frame every 12.88 ms instead of a single 19.23 ms frame.